



ECA304

PROGRAMAÇÃO

ORIENTADA A OBJETOS

Alexandre H. MOREIRA

CLASSES E OBJETOS - INTRODUÇÃO



Introdução

Estudar programação orientada a objetos (POO) é importante por várias razões:

- **Modelagem mais próxima da realidade:**

A POO permite representar entidades do mundo real como "objetos" com **atributos** (características) e **métodos** (comportamentos). Isso facilita a compreensão e o mapeamento de problemas reais para soluções computacionais.



Introdução

- **Organização e reutilização do código:**

A POO promove a **modularização** do código, facilitando:

- **Reutilização** de classes em diferentes partes do projeto ou em projetos futuros.
- **Manutenção** e atualização de sistemas grandes e complexos.
- Redução de **repetição de código** (princípio DRY: Don't Repeat Yourself).



□ Encapsulamento:

Objetos mantêm seus dados protegidos por meio do encapsulamento, ou seja, os detalhes internos ficam escondidos do resto do sistema. Isso:

- Aumenta a **segurança**.
- Reduz o risco de **erros** acidentais ao manipular dados internos.

□ Herança:

A **herança** permite criar novas classes a partir de classes existentes, evitando a duplicação de código e promovendo uma estrutura hierárquica e reutilizável.



Introdução

□ **Polimorfismo:**

O **polimorfismo** permite que diferentes objetos respondam de maneiras diferentes a uma mesma chamada de método. Isso dá flexibilidade para escrever código mais genérico e escalável.

□ **Amplamente usada no mercado:**

A POO é o paradigma dominante em muitas linguagens modernas (Java, C++, Python, C#, etc.) e em sistemas grandes.

□ **Base para outros conceitos:**

A POO é também um pré-requisito para aprender conceitos como design patterns.



Comparação

□ **Procedural:**

```
def apresentar(nome, idade):  
    print(f"Olá, meu nome é {nome} e tenho  
    {idade} anos.")
```

```
nome1 = "Alice"  
idade1 = 25
```

```
nome2 = "Bruno"  
idade2 = 30
```

```
apresentar(nome1, idade1)  
apresentar(nome2, idade2)
```



□ Orientado a objetos:

```
class Pessoa:
```

```
    def __init__(self, nome, idade):
```

```
        self.nome = nome
```

```
        self.idade = idade
```

```
    def apresentar(self):
```

```
        print(f"Olá, meu nome é {self.nome} e tenho {self.idade} anos.")
```

```
# Criando objetos (instâncias)
```

```
p1 = Pessoa("Alice", 25)
```

```
p2 = Pessoa("Bruno", 30)
```

```
# Chamando métodos
```

```
p1.apresentar()
```

```
p2.apresentar()
```



Introdução

Classes e Objetos - Introdução

A Programação Orientada a Objetos (POO) tem **quatro pilares fundamentais**. Esses pilares são os princípios básicos que sustentam a forma como o paradigma orientado a objetos funciona. São eles:



1. Abstração



2. Encapsulamento



3. Herança



4. Polimorfismo



□ **Abstração:**

Consiste em **destacar apenas as características essenciais** de um objeto, ignorando os detalhes irrelevantes.

Exemplo:

Um carro pode ser representado por atributos como modelo, cor e velocidade, e métodos como `acelerar()` e `frear()`. Não é necessário modelar o funcionamento do motor para simular a condução.



□ Encapsulamento:

É o princípio de **esconder os dados internos do objeto**, permitindo o acesso e modificação apenas por meio de métodos específicos (getters/setters, por exemplo).

□ Herança:

Permite que uma classe **herde atributos e métodos de outra classe**, promovendo **reutilização de código e especialização**.

Exemplo:

Uma **classe Animal** pode ter um método **fazerSom()**, e **classes filhas** como **Cachorro** e **Gato** podem herdar esse método e implementar sons diferentes.



□ Polimorfismo:

Significa "muitas formas". Permite que objetos de diferentes classes **respondam de forma diferente à mesma mensagem** (método).

Exemplo:

Se várias classes têm um método chamado `desenhar()`, você pode chamar `desenhar()` em qualquer objeto sem saber sua classe exata — cada um executará sua própria versão do método.



Usando POO – Criando a Classe

Classes e Objetos - Introdução

```
#Criando uma classe
class Pessoa:
    #Atributos
    nome = ""
    idade = 0

#Criando um objeto ou uma instância
pessoa1 = Pessoa()

#Acessando seus atributos
pessoa1.nome = "Fulano de Tal"
pessoa1.idade = 33

print(f"Olá, meu nome é {pessoa1.nome} e tenho {pessoa1.idade} anos.")
```

Olá, meu nome é Fulano de Tal e tenho 33 anos.



Usando POO - Atributos

Classes e Objetos - Introdução

```
#Criando uma classe
class Pessoa:
    #Atributos
    nome = ""
    idade = 0

#Criando um objeto ou uma instância
pessoa1 = Pessoa()

#Acessando seus atributos
pessoa1.nome = "Fulano de Tal"
pessoa1.idade = 33
pessoa1.CPF = "123.456.789-00"

print(f"Olá, meu nome é {pessoa1.nome}, tenho {pessoa1.idade} anos e CPF: {pessoa1.CPF}.")
```

Até podemos criar novos atributos fora da classe, mas isso não é uma boa prática!

Olá, meu nome é Fulano de Tal, tenho 33 anos e CPF: 123.456.789-00.



Usando POO - Métodos

Classes e Objetos - Introdução

```
#Criando uma classe
class Pessoa:
    #Atributos
    nome = ""
    idade = 0

    #Métodos
    def atribuirNome(self, nome):
        self.nome = nome

    def atribuirIdade(self, idade):
        self.idade = idade

    def exibirInfo(self):
        print(f"Olá, meu nome é {self.nome} e tenho {self.idade} anos.")

#Criando um objeto ou uma instância
pessoa1 = Pessoa()

#Acessando seus métodos
pessoa1.atribuirNome("Fulano de Tal")
pessoa1.atribuirIdade(33)
pessoa1.exibirInfo()
```



Usando POO - Construtor da classe

Classes e Objetos - Introdução

```
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def atribuirNome(self, nome):
        self.nome = nome

    def atribuirIdade(self, idade):
        self.idade = idade

    def exibirInfo(self):
        print(f"Olá, meu nome é {self.nome} e tenho {self.idade} anos.")

pessoa1 = Pessoa("Fulano de Tal", 33)

pessoa1.exibirInfo()
```

Olá, meu nome é Fulano de Tal e tenho 33 anos.



Usando POO - Construtor da classe

Classes e Objetos - Introdução

□ Os métodos ainda funcionam?

```
pessoa1 = Pessoa("Fulano de Tal", 33)
pessoa1.atribuirIdade(99)
pessoa1.exibirInfo()
```

Olá, meu nome é Fulano de Tal e tenho 99 anos.

□ E isso?

```
pessoa1 = Pessoa()
```

Traceback (most recent call last):

File "c:\Users\Alexandre\Desktop\Untitled-1.py", line 17, in <module>

```
    pessoa1 = Pessoa()
               ^^^^^^^
```

TypeError: Pessoa.__init__() missing 2 required positional arguments: 'nome' and 'idade'



Usando POO - Construtor da classe

Classes e Objetos - Introdução

```
class Pessoa:
    def __init__(self, nome = "", idade = None):
        self.nome = nome
        self.idade = idade

    def atribuirNome(self, nome):
        self.nome = nome

    def atribuirIdade(self, idade):
        self.idade = idade

    def exibirInfo(self):
        print(f"Olá, meu nome é {self.nome} e tenho {self.idade} anos.")

pessoa1 = Pessoa()

pessoa1.exibirInfo()
```

Olá, meu nome é e tenho None anos.



Modelando Objetos

Classes e Objetos - Introdução

- Modelar um objeto significa **identificar e definir suas características e comportamentos** de forma que ele represente bem um elemento do mundo real (ou do problema que estamos resolvendo)
- **Unified Modeling Language (UML):**
 - É a linguagem mais comum para modelar sistemas orientados a objetos.
 - Um **diagrama de classes** mostra:
 - As classes do sistema
 - Seus atributos e métodos
 - Relações entre classes (herança, associação, etc.)



Modelando Objetos

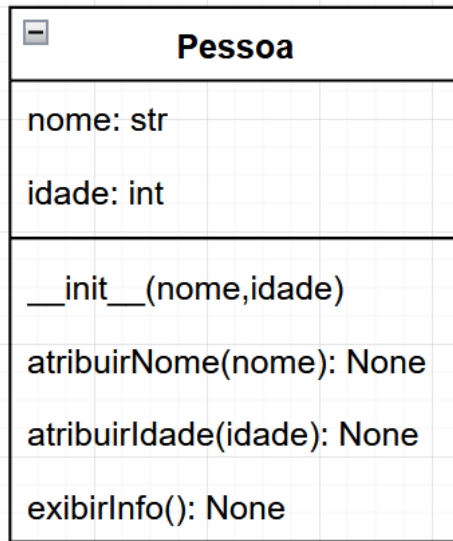
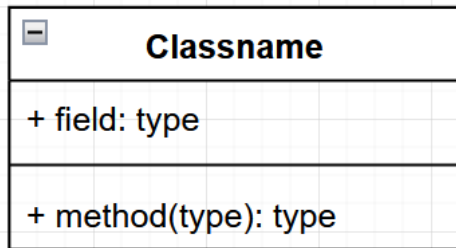
Classes e Objetos - Introdução

- Modelar um objeto significa **identificar e definir suas características e comportamentos** de forma que ele represente bem um elemento do mundo real (ou do problema que estamos resolvendo)
- **Unified Modeling Language (UML):**
 - É a linguagem mais comum para modelar sistemas orientados a objetos.
 - Um **diagrama de classes** mostra:
 - As classes do sistema
 - Seus atributos e métodos
 - Relações entre classes (herança, associação, etc.)



Modelando Objetos

- Representação típica de uma classe:





EXERCÍCIO PARA ENTREGA

A CLASSE POKÉMON



Exercício para entrega

- ❑ Criar uma classe Pokémon assim como a representada no diagrama UML. Não se esqueça de incluir um teste para sua classe (criar o objeto, acessar atributos e métodos, etc.):

